

Pokémon Database

Introduction

This project focuses on designing a relational database to organize and analyze Pokémon data. Pokémon have many different attributes, such as types, stats, generations, egg groups, colors, and level-up groups. Because these attributes are connected in different ways, the data can become difficult to manage if it is stored in one large table. The purpose of this database is to organize Pokémon data efficiently so users can search, compare, and analyze Pokémon based on different characteristics. This database can be useful for players, analysts, and researchers who are interested in exploring Pokémon data.

This project is important because Pokémon data contains many repeated and connected pieces of information. For example, many Pokémon share the same type, egg group, generation, or level-up group. If all of this information were stored in one large table, the database would become repetitive and harder to update. A relational database solves this problem by separating the data into related tables and connecting them through primary keys and foreign keys. This makes the database easier to manage, more accurate, and more useful for answering specific questions.

Database Functionality and Key Questions

The database allows users to search for Pokémon based on specific conditions, such as level-up group, attack stat, color, generation, egg group, and typing. For example, a user could search for brown Pokémon from Generation 1 or Generation 3 that belong to the Mineral or Field egg groups, are in the Erratic level-up group, and have Ghost, Ground, or Bug typing. The results can then be sorted by health

points in ascending order to identify the Pokémon with the lowest HP that meets those conditions, such as Shedinja.

This database is designed to answer questions such as: Which Pokémon has the highest total stats? Which Pokémon are in a specific egg group? Which Pokémon are dual-type? Which Pokémon has the higher Speed stat in a battle matchup? How many Pokémon were added in each generation? These questions show how the database can support both gameplay decisions and general Pokémon data analysis for competitive or casual players.

Another key function of this database is that it allows users to combine multiple conditions in one query. Instead of only searching for one attribute at a time, users can search across several categories at once. For example, a user can filter Pokémon by color, generation, egg group, type, level-up group, and stat value in a single query. This makes the database more useful than a basic spreadsheet because the data is connected through relationships. A user can answer more complex questions, such as finding Pokémon that meet a very specific battle or breeding requirement.

The database also supports comparison-based questions. For battle strategy, players may want to compare Speed, Attack, Defense, or total stats between Pokémon. For breeding or collection purposes, users may want to find all Pokémon in the same egg group or all Pokémon that share a certain type. These questions show that the database is designed not just for storing information, but also for helping users make decisions based on the data.

Database Design

The database stores information about Pokémon and their related attributes. Each Pokémon has a unique Pokédex number, which serves as the primary key in the Pokémon table. The Pokémon table also stores each Pokémon's name, color, generation ID, and level-up group ID. This allows each Pokémon to be identified clearly while still connecting it to other tables through foreign keys.

The database includes several related tables: Pokémon, Types, Stats, Generation, Egg Groups, and Level Up Group. Since a Pokémon can have one or two types, the many-to-many relationship between Pokémon and Types is handled through the **Pokemon_Type** bridge table. This avoids storing repeated type names directly in the Pokémon table and allows the same type to be connected to many different Pokémon.

The same structure is used for stats and egg groups. The Stats table stores the names of the stats, such as Health Points, Attack, Defense, Special Attack, Special Defense, and Speed. The **Pokemon_Stat** table connects each Pokémon to each stat and stores the numerical value for that stat. This design is necessary because the value depends on both the Pokémon and the stat. For example, Speed by itself is only a stat category, but Aerodactyl's Speed value is 130. Therefore, the value belongs in the relationship table instead of directly in the Stats table.

Egg groups are also handled with a bridge table because a Pokémon can belong to more than one egg group, and each egg group can contain many Pokémon. The **Pokemon_EggGroup** table represents this many-to-many relationship. Generation and Level Up Group are modeled differently because each Pokémon belongs to one generation and one level-up group. These are stored as foreign keys in the Pokémon table. This structure keeps the database normalized, avoids repeated data, and allows users to query Pokémon across multiple attributes efficiently.

The E-R diagram in the Appendix shows the main entities and relationship sets, including Pokémon, Types, Stats, Generation, Level Up Group, and Egg Groups. The relational diagram shows how the E-R design was converted into tables, primary keys, foreign keys, and bridge tables. Together, these diagrams explain both the conceptual design and the final relational structure of the database.

Overall, this design follows the purpose of a relational database by keeping related information separated but connected. The bridge tables are especially important because they allow the database to represent real Pokémon relationships accurately, such as Pokémon having multiple types, multiple stats,

and multiple egg groups. This structure makes the database easier to update and prevents the same information from being repeated across many rows.

Data Sources

The data used in this project was obtained from publicly available Pokémon datasets, including Pokémon Database, Serebii, and PokeAPI GitHub data files. These sources provide detailed information about Pokémon stats, types, egg groups, level-up groups, colors, and other attributes. The data was first imported into staging tables and then transformed into the final normalized database structure.

After collection, the data was imported into staging tables before being moved into the final database tables. This step was useful because the original datasets were not in the same structure as the final database. For example, some raw data files listed Pokémon types and stats in one flat table. However, the final database separates those values into multiple related tables. Using staging tables made it easier to clean the data, match type names, assign stat values, update colors, connect egg groups, and assign level-up groups.

This process also helped ensure that the database followed a normalized structure. Instead of keeping the raw data exactly as it appeared in the source files, the data was transformed into separate tables that matched the E-R and relational diagrams. This made the final database more organized and reduced repeated information. The staging tables were temporary and were only used to help import and transform the data.

Data Analysis

The database can help Pokémon players make decisions during battles by comparing important stats between Pokémon. For example, if a player wants to know whether Aerodactyl or Greninja is more likely to attack first, they can compare their Speed stats because the Pokémon with the higher Speed usually moves first. The query results show that Aerodactyl has a Speed stat of 130, while

Greninja has a Speed stat of 122. Based on this result, Aerodactyl would generally move before Greninja, which could help a player decide which Pokémon to use in battle.

The database can also help users identify the correct egg group for a Pokémon. This is useful for breeding because a player may want to breed a Pokémon, such as Pikachu, with another compatible Pokémon in the same egg group. The query can find Pikachu's egg group and then list all other Pokémon that belong to that same group. In the example result, Pikachu belongs to the Fairy egg group, so the database returns Pokémon that are also in the Fairy egg group, such as Alcremie, Aromatisse, Audino, Azumarill, and Clefairy. This shows that the database is useful not only for looking up individual Pokémon attributes, but also for finding relationships between Pokémon based on shared characteristics.

The complex filtering query also demonstrates the strength of the database design. Because the database separates Pokémon, stats, types, egg groups, generations, and level-up groups into related tables, the user can combine all of these categories in one query. For example, the database can search for Pokémon based on color, generation, egg group, type, level-up group, and health points at the same time. This would be much harder to manage in a simple spreadsheet because the information would either be repeated or stored in a less flexible way. The relational database structure makes these advanced searches possible.

One example of this is a query that searches for blue or pink Pokémon from Kanto or Johto (generation 1 or 2) that belong to the Water 1, Monster, or Field egg groups, have Psychic, Water, or Ground typing, are in the Medium-Fast level-up group, and are sorted by speed stat in ascending order. The result identifies Pokémon such as Slowpoke, and Wooper which have very low speed stats compared to most Pokémon. This finding shows that the database can be used to locate Pokémon with very specific characteristics. It also demonstrates that the database can support more than simple lookups; it can help users discover patterns and compare Pokémon based on multiple conditions.

Overall, the findings show that the database can support both casual and strategic uses. A casual user may use it to look up a Pokémon's type, color, generation, or

egg group. A competitive player may use it to compare battle stats, identify faster Pokémon, or search for Pokémon with specific stat advantages. A data analyst could use the database to study patterns across generations, types, egg groups, or level-up groups. By organizing the data into related tables, the database turns a large collection of Pokémon information into a system that can answer detailed and meaningful questions.

In addition to the examples shown, the database can be expanded to answer many other questions about Pokémon data. For example, users could compare average stats across generations, identify which types appear most often, or find which egg groups contain the strongest Pokémon based on total stat values. Because the data is organized into separate but connected tables, these new questions can be answered without redesigning the database. This shows that the database is flexible and can support future analysis beyond the queries included in this report. The structure also makes it easier to add new Pokémon or update existing Pokémon information because shared data, such as type names and stat names, is stored only once in lookup tables. Overall, this project shows how relational database design can turn a large and complicated dataset into a useful system for searching, comparing, and analyzing Pokémon information in a clear and organized way.

References

PokeAPI. (n.d.). *PokeAPI/pokeapi: The Pokémon API* [GitHub repository]. GitHub. Retrieved May 1, 2026, from <https://github.com/PokeAPI/pokeapi>

PokeAPI. (n.d.). *pokemon_egg_groups.csv* [Data set]. GitHub. Retrieved May 1, 2026, from https://raw.githubusercontent.com/PokeAPI/pokeapi/master/data/v2/csv/pokemon_egg_groups.csv

PokeAPI. (n.d.). *egg_groups.csv* [Data set]. GitHub. Retrieved May 1, 2026, from https://raw.githubusercontent.com/PokeAPI/pokeapi/master/data/v2/csv/egg_groups.csv

PokeAPI. (n.d.). *growth_rates.csv* [Data set]. GitHub. Retrieved May 1, 2026, from https://raw.githubusercontent.com/PokeAPI/pokeapi/master/data/v2/csv/growth_rates.csv

Pokémon Database. (n.d.). *Pokémon plain text exporter*. Retrieved May 1, 2026, from <https://pokemondb.net/tools/text-list>

Serebii.net. (n.d.). *Serebii.net*. Retrieved May 1, 2026, from <https://www.serebii.net/>

Appendix

The Appendix includes the E-R diagram, relational schema diagram, SQL queries, and query result screenshots used to support the database design and analysis.

Figure 1. E-R diagram for the Pokémon database.

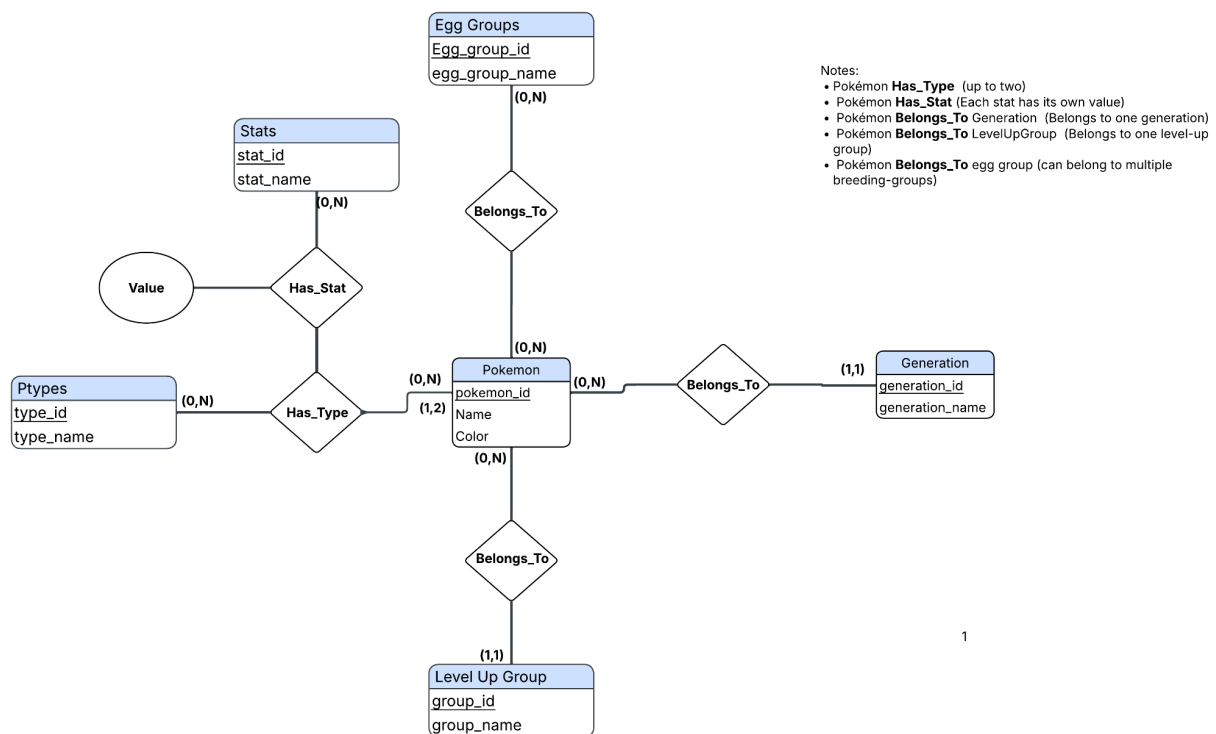


Figure 2. Relational schema diagram for the Pokémon database.

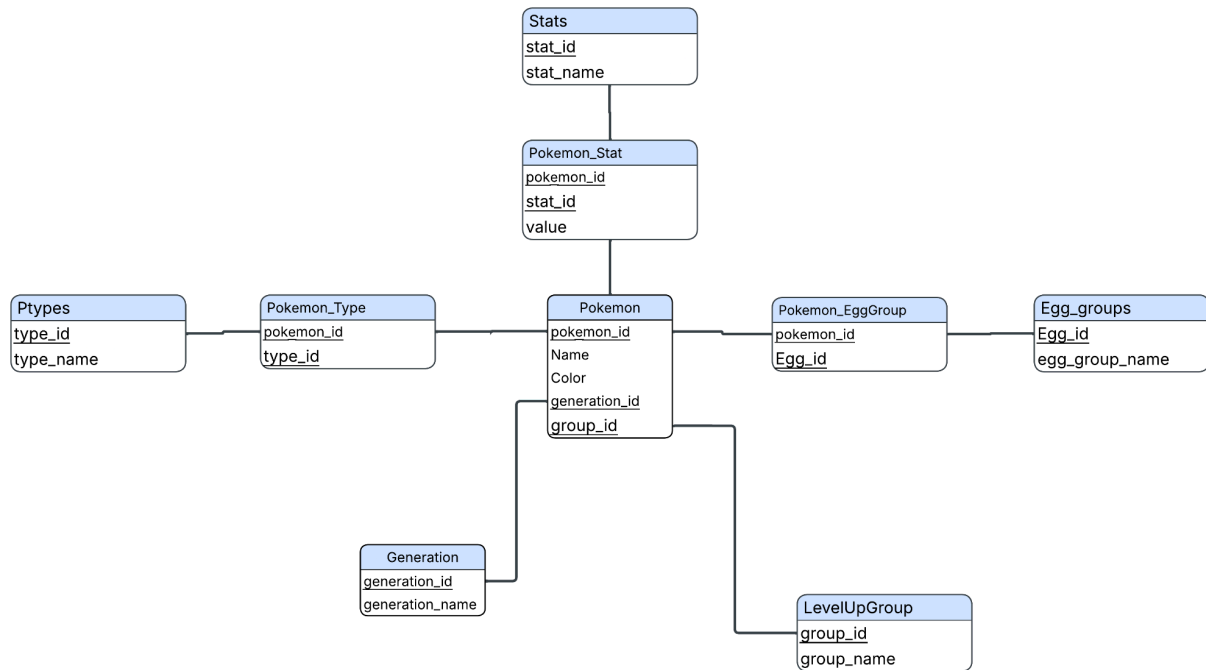


Figure 3. Query result comparing Aerodactyl and Greninja's Speed stats.

```

412     #Checking to see if Aerodactyl is faster than Greninja
413 •   SELECT p.pokemon_id, p.name, ps.value AS speed
414     FROM pokemon p
415     JOIN Pokemon_Stat ps ON p.pokemon_id = ps.pokemon_id
416     WHERE ps.stat_id = 6
417     AND p.name IN ('Greninja', 'Aerodactyl')
418     ORDER BY ps.value DESC;
    
```

Result Grid Filter Rows: Export: Wrap Cell Content:			
	pokemon_id	name	speed
▶	142	Aerodactyl	130
	658	Greninja	122

Figure 4. Query result showing Pokémon in Pikachu's egg group.

```
397 # Finding what egg group Pikachu belongs in and showing all
398 # the Pokémon in the same group
399 • SELECT DISTINCT p2.pokemon_id, p2.name, e.egg_group_name
400 FROM pokemon p1
401 JOIN Pokemon_EggGroup pe1
402     ON p1.pokemon_id = pe1.pokemon_id
403 JOIN egg_groups e
404     ON pe1.egg_id = e.egg_id
405 JOIN Pokemon_EggGroup pe2
406     ON e.egg_id = pe2.egg_id
407 JOIN pokemon p2
408     ON pe2.pokemon_id = p2.pokemon_id
409 WHERE p1.name = 'Pikachu'
410 ORDER BY e.egg_group_name, p2.name;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 			
	pokemon_id	name	egg_group_name
▶	869	Alcremie	Fairy
	683	Aromatisse	Fairy
	531	Audino	Fairy
	184	Azumarill	Fairy
	242	Blissey	Fairy
	286	Breloom	Fairy
	703	Carbink	Fairy
	351	Castform	Fairy
	113	Chansey	Fairy
	421	Cherrim	Fairy
	420	Cherubi	Fairy
	36	Clefable	Fairy

Figure 5. SQL queries used for database analysis.

```

377 # lowest health Pokémon with multiple conditions (using names)
378 • SELECT DISTINCT p.pokemon_id, p.name, ps.value AS health_points
379 FROM pokemon p
380 JOIN Pokemon_Stat ps ON p.pokemon_id = ps.pokemon_id
381 JOIN stat s ON ps.stat_id = s.stat_id
382 JOIN Pokemon_EggGroup pe ON p.pokemon_id = pe.pokemon_id
383 JOIN egg_groups e ON pe.egg_id = e.egg_id
384 JOIN Pokemon_Type pt ON p.pokemon_id = pt.pokemon_id
385 JOIN ptypes t ON pt.type_id = t.type_id
386 JOIN generation g ON p.generation_id = g.generation_id
387 JOIN levelupgroup lg ON p.group_id = lg.group_id
388 WHERE p.color = 'Brown'
389 AND s.stat_name = 'Health Points'
390 AND g.generation_name IN ('Kanto', 'Hoenn')
391 AND e.egg_group_name IN ('Mineral', 'Field')
392 AND t.type_name IN ('Ghost', 'Ground', 'Bug')
393 AND lg.group_name = 'Erratic'
394 ORDER BY ps.value ASC

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	pokemon_id	name	health_points
▶	292	Shedinja	1